1. Práctica 4: Entrada / Salida por interrupciones - UART RX y botones

1.1. Objetivos

- Aprender qué son las interrupciones, cómo funcionan, y sus características.
- Programar el controlador de interrupciones para la recepción de datos con UART.
- Programar el controlador de interrupciones para entrada con botón.

1.2. Fundamentos teóricos

1.2.1. Configuración de las interrupciones en el procesador

Para responder de manera rápida y consistente a eventos en un microcontrolador, es buena idea utilizar una *interrupción hardware* para ejecutar un pequeño fragmento de código al detectar dichos eventos. En esta práctica, vamos a ver cómo funciona el controlador de interrupciones en STM32, que puede configurarse para ejecutar ciertas funciones al ocurrir determinados eventos.

Cada procesador tiene su propia manera de tratar interrupciones. En la familia STM32, y en concreto de **STM32F723IE**, se utilizan dos periféricos diferentes para su gestión:

- `NVIC` : El (*Nested Vectored Interrupt Controller*) `pm0253 4.2` es el gestor de interrupciones interno del procesador Cortex-M7, núcleo dentro del **STM32F723IE**. Permite la programación de hasta 240 interrupciones, con control de prioridad en cada una de ellas. En el caso de nuestro procesador, **STM32F723IE**, están disponibles unas 100 (`rm0431 9.1.2`), aunque utilizaremos nada más que un par de ellas.
- `EXTI` : (*EXtended Interrupt Controller*). Su propósito es extender el número de interrupciones disponibles a los programadores a través de eventos externos al propio procesador. Cada periférico interno tiene sus propias líneas de interrupción en el `NVIC` , como puede verse en `rm0431 9.1.2` , sin embargo, para los eventos en pines de E/S (a través de GPIO), la configuración de la interrupción se hace desde el `EXTI` .

  En la familia STM32, los pines GPIO están agrupados (PA0, PB0, PC0...). El periférico `EXTI` no tiene una línea directa para cada pin individual de la placa, sino que usa líneas compartidas (la línea `EXTI0` gestiona PA0, PB0, PC0, etc. a través de un multiplexor en el `SYSCFG`). Por tanto, no se pueden usar, por ejemplo, PA0 y PB0 como interrupciones independientes simultáneamente.

Adicionalmente, los periféricos externos, aunque estén conectados a las líneas de interrupción, deben ser configurados para generar las señales de interrupción necesarias.

En resumen, estos procesadores tienen interrupciones internas a través del `NVIC` , que hay que activar, además de realizar la configuración de los periféricos y, algunos casos como el GPIO, configurar `EXTI` para recibir sus interrupciones. En esta práctica, configuraremos las interrupciones de `UART` directamente desde `NVIC` , y las de `GPIO` utilizando también `EXTI` . Podemos ver un diagrama de las interconexiones entre todo en la Figura 1.

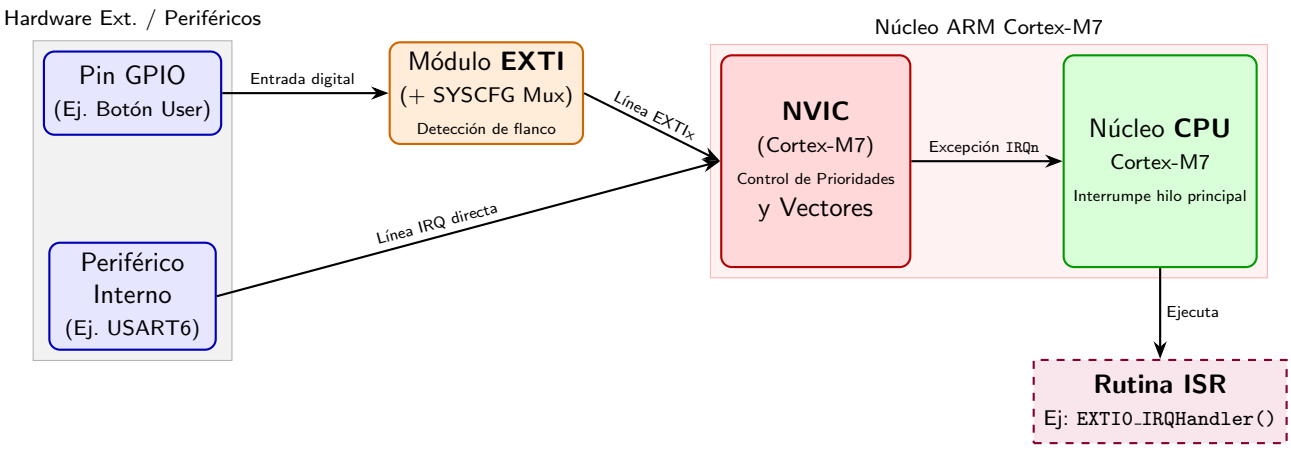


Figura 1: Flujo de interrupciones en el microcontrolador STM32F723E.

1.2.2. Gestión de interrupciones mediante funciones

Cuando se produce una interrupción, el procesador debe pausar su ejecución y saltar a la dirección de memoria donde se encuentra la rutina de respuesta o ISR (Interrupt Service Routine). Como la ubicación física de estas funciones en la memoria Flash no es fija (depende de la compilación), el procesador utiliza una Tabla de Vectores de Interrupción (Vector Table).

Esta tabla es un array de punteros ubicado al inicio de la memoria (por defecto desde `0x00000000`), donde cada posición está reservada para un evento concreto:

- La posición `0x00000000` almacena el valor inicial del puntero de pila (MSP).
- La dirección `0x00000004` contiene el vector de Reset.
- Direcciones posteriores contienen los vectores o punteros a las rutinas de interrupción, como por ejemplo `0x0000015C` que contiene el puntero a la rutina de la `USART6` (`rm0431 9.1.2`).

Para evitarnos la tarea de configurar esta tabla manualmente, el código de inicio del procesador (`Startup/startup_stm32f723iekx.s`) la genera automáticamente durante el arranque. Por defecto, todas las entradas de la tabla apuntan a un bucle infinito genérico (`Default_Handler`) mediante el atributo `.weak`. Si definimos en nuestro código C una función con el nombre exacto de la interrupción (por ejemplo, `USART6_IRQHandler`), el enlazador (linker) la sobrescribirá automáticamente, redirigiendo la tabla hacia nuestra rutina. Podemos ver este proceso en Figura 2.

1.3. Desarrollo de la práctica

Como en prácticas anteriores, el objetivo es seguir extendiendo nuestro BSP con las funciones necesarias como para controlar los periféricos y funcionalidades que utilizaremos en esta práctica. En este caso, debemos hacer varias cosas:

1. Apúntate las direcciones base de los periféricos `NVIC` en `pm0253 4.2`, y `EXTI` y `SYSCFG` en `rm0431 1.6.2`, crea además las estructuras de datos con las que accederás a los registros. Básate, respectivamente, en `pm0253 4.2` para `NVIC` (Ten en cuenta que los reigstros

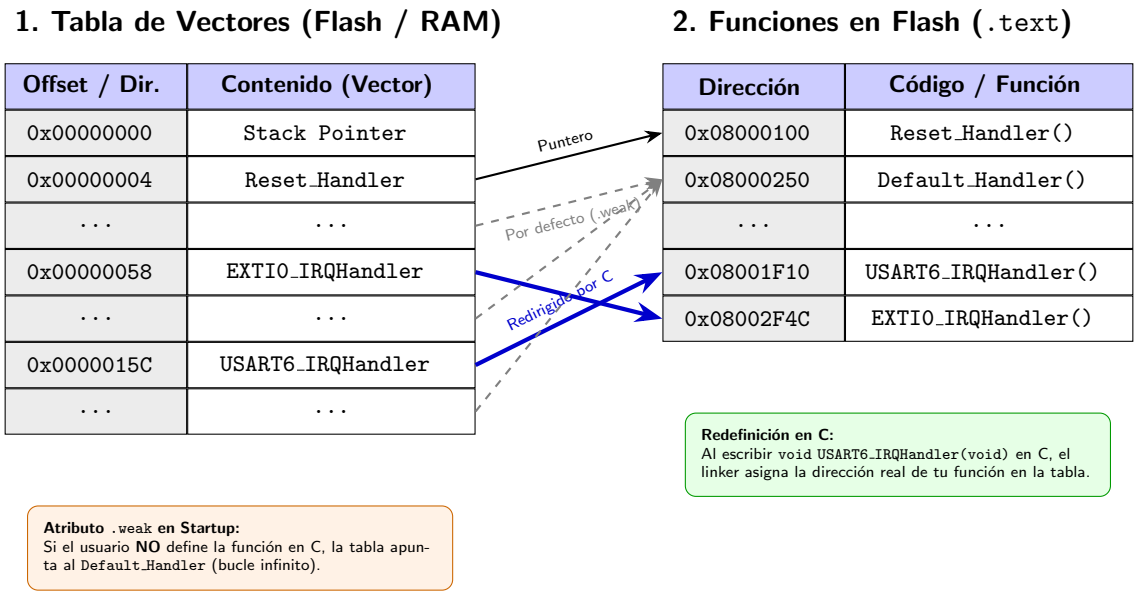


Figura 2: Mapeo de la Tabla de Vectores de Interrupción a la memoria del programa.

no están consecutivos), en `rm0431 10.7` y posteriores para `EXTI`, y en `rm0431 7.2.8` para `SYSCFG`. Crea además las variables para manejarlos. Puedes basarte en el siguiente código:

```
#define EXTI_BASE_ADDR      0x40013C00UL
//... completar para NVIC y SYSCFG

typedef struct
{
    volatile uint32_t ISER[8U];               /*!< Offset: 0x000 (R/W)
        Interrupt Set Enable Register */
    uint32_t RESERVED0[24U];
    //... completar
}  NVIC_TypeDef;

typedef struct
{
    volatile uint32_t MEMRMP;       /*!< SYSCFG memory remap register,
                          Address offset: 0x00      */
    //... completar
} SYSCFG_TypeDef;

typedef struct
{
    volatile uint32_t IMR;    /*!< EXTI Interrupt mask register,
                  Address offset: 0x00 */
    //... completar
} EXTI_TypeDef;

#define EXTI                ((EXTI_TypeDef *) EXTI_BASE_ADDR)
//... completar para NVIC y SYSCFG
```

2. Debemos activar el puerto y pin correspondiente al botón. Revisando la documentación, en `um2140 Table 5` nos dice que el botón está enchufado a la función wake-up del procesador, y en `um2140 Table 19` que dicha función `SYS_WKUP1` está en el PA0. Por tanto, lo primero es configurar este pin como entrada, de tipo *push-pull* con el *pulldown* activado, y a velocidad alta. No te olvides de activar también el reloj de GPIOA a través de `AHB1ENR`.

3. A continuación, hay que activar también el pin de transmisión de la UART. En um2140 Table 19 podemos observar que la función USART6_RX está conectada al pin PC7. Tenemos que configurarlo en modo *alternativo* con la función 8 según nos indica ds11853 Table 12. Adicionalmente, configurarlo en modo *push-pull* con resistencia de *pullup* y velocidad alta.

4. Finalmente, configuramos la UART para que también funcione en modo recepción y tenga activas las interrupciones de recepción. Ambas cosas se pueden hacer desde el registro CR1 (rm0431 27.8.1). Crea funciones en el BSP y define las constantes necesarias. Puedes hacer algo de este estilo:

```
//en main.c
UART_RX_MODE_SET(USART6, UART_RX_ENABLE);
UART_RX_INT_SET(USART6, UART_RX_INT_ENABLE);


//en bsp.h
// RECEIVE enable
typedef enum {
    UART_RX_DISABLE = 0x00,
    UART_RX_ENABLE  = 0x01
} UART_RX_Mode_t;
#define UART_RX_MODE_MASK  0b1
#define UART_RX_MODE_SHIFT 2

static inline void UART_RX_MODE_SET(volatile USART_TypeDef *USARTx,
    UART_RX_Mode_t mode) {
    //... completar tocar el bit adecuado de CR1
}

//... completar hacer una función similar para RX INT enable
```

5. Ahora ya podemos comenzar a activar las interrupciones. Lo primero es activar el reloj para la configuración del sistema en APB2ENR (rm0431 5.3.14), para lo cual previamente deberemos definir los campos del registro.

```
//en bsp.h
#define RCC_APB2EN_BASE_ADDR 0x40023844 //APB2 clock control register

//Structure for Reset Control peripheral register APB1
typedef union {
    volatile uint32_t reg;

    // 2. Access individual bits or bit-groups for the register above
    struct {
        volatile uint32_t TIM1EN        : 1;  // Bit 0:
        volatile uint32_t TIM8EN        : 1;  // Bit 1:
        volatile uint32_t RES1          : 2;  // Bit 2-3:
        //... completar
    } bits;
} RCC_APB2R_TypeDef;

#define RCC_APB2ENR         ((RCC_APB2R_TypeDef *) RCC_APB2EN_BASE_ADDR)
```

A continuación, debemos configurar la interrupción externa EINT0 (que multiplexa todos los pines 0) para que trabaje con el puerto A (recordemos que el botón está en PA0). Esto se hace desde el registro EXTICR de SYSCFG (rm0431 7.2.3). Finalmente, hay que activar la interrupción externa 0 desde EXTI a través del registro IMR (rm0431 10.9.1) en modo *rising trigger* en el registro RTSR (rm0431 10.9.3).

```
//... completar en bsp.h crear y rellenar estas funciones
```

```
static inline void SYSCFG_EINT_MAP_PORT(uint32_t eintno, uint32_t portno) {
    //... completar modificar SYSCFG->EXTICR
}

static inline void EXTI_ENABLE_RISING_TRIGGER(uint32_t eintno, uint32_t
    enable) {
    //... completar activar/desactivar EXTI->RTSR
}

static inline void EXTI_UNMASK_INTERRUPT(uint32_t eintno, uint32_t unmask)
    {
    //... completar activar/desactivar EXTI->IMR
}


//en main.c
RCC_APB2ENR->bits.SYSCFGEN = 1;
//Activar EINT0 PUERTO A
SYSCFG_EINT_MAP_PORT(0, 0); //port a (0) to eint0
EXTI_ENABLE_RISING_TRIGGER(0, 1);   //enable on port a
EXTI_UNMASK_INTERRUPT(0, 1);        //unmask on port a
```

6. Ya tendríamos con esto el pin PA0 generando una interrupción en `EXTI0`, pero el procesador aún no responde a estas interrupciones. Adicionalmente, necesitamos activar la interrupción de la `UART`. Ambas cosas se hacen desde el `NVIC` en el registro `ISER` (`pm0253 4.2.2`). Para saber qué bits activar, debemos consular la numeración de las líneas de interrupción en `rm0431 9`.

```
//en bsp.h
//... completar buscar los números de línea
#define EXTI0_LINE_IRQn
#define USART6_INT_IRQn

static inline void NVIC_ENABLE_INT(uint32_t line) {
    //... completar modificar el registro NVIC->ISER
}

//en main.c
NVIC_ENABLE_INT(EXTI0_LINE_IRQn);
NVIC_ENABLE_INT(USART6_INT_IRQn);
```

7. Por último, necesitamos crear dos funciones que respondan a sendas interrupciones. En este caso, la función `USART6_IRQHandler` que responda a la `USART6`, y `EXTI0_IRQHandler` que responda al botón.

8. En el caso de la primera, `USART6_IRQHandler`, deberemos comprobar primero que la interrupción ha sido generada por recepción, consultando el bit `RXNE` del registro `ISR` de la `UART` (`rm0431 27.8.8`). Si es así, podremos leer el registro `RDR` (`rm0431 27.8.10`) para consultar el dato recibido, lo cual a su vez limpiará la interrupción. Dentro de la interrupción, para darle funcionalidad, podemos por ejemplo enviar de vuelta el dato leído al ordenador.

```
void USART6_IRQHandler(void)
{
    //... completar comprobar si el flag de interrupción está activo
    if () {
        //... completar leer el byte y retransmitirlo
    }
}
```

9. En el caso de la segunda `EXTI0_IRQHandler`, el proceso es diferente. Primero, debemos comprobar que la interrupción está pendiente desde el registro `PR` de `EXTI` (`rm0431 10.9.6`). A continuación, habrá que borrar dicho bit, escribiendo de nuevo en el registro, como indica la documentación. En esta función, por ejemplo, se puede poner como funcionalidad el dar la vuelta al LED que ya teníamos configurado anteriormente, para comprobar que funciona.

```
void EXTI0_IRQHandler(void)
{
    //... comprobar en EXTI->PR si la línea 0 generó interrupción
    if () {
        //... limpiar línea 0 escribiendo la posición 0 de EXTI->PR
        //... completar funcionalidad
    }
}
```

1.4. Para hacer más

Intenta cambiar "en caliente" la función que gestiona una interrupción, por ejemplo la del botón, modificando el puntero de las direcciones de memoria a las que salta el `NVIC`.